Mini Juego Piedra, Papel o Tijera con Persistencia: Struct y Flujos de Archivo para Memoria Volátil y Disco No Volátil

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B

Prof. Mansilla Muñoz York Elías

2026

Resumen

Se desarrolla un juego de Piedra, Papel o Tijera contra inteligencia artificial donde cada partida se modela con la estructura *Partida* y se persiste en disco con flujo de salida en modo adjunto (*ios::app*), recuperándose con flujo de entrada que lee formato CSV. Se contrasta la memoria RAM volátil —los datos se pierden al cerrar el programa— con el disco no volátil —el archivo *historial_partidas.txt* permanece—. Las pruebas jugando, cerrando y reabriendo el ejecutable demuestran que la opción 2 continúa mostrando el historial. Se analiza el rol de *cin.ignore()* y de la estructura para cohesión. La técnica es extrapolable a registros de sensores y stock del Proyecto Integrador Anual.

Palabras clave: persistencia, fstream, ofstream, ifstream, ios::app, struct, memoria volátil, disco no volátil, CSV

Abstract

A Rock, Paper, Scissors game versus artificial intelligence is developed where each match is modeled with the *Partida* structure and persisted to disk with an output stream in append mode (*ios::app*), recovered with an input stream reading CSV format. Volatile RAM —data lost on close— is contrasted with non-volatile disk —*historial_partidas.txt* persists—. Tests playing, closing, and reopening the executable show that option 2 still displays the history. The role of *cin.ignore()* and of the structure for cohesion is analyzed. The technique is extrapolable to sensor logs and stock records of the Annual Integrative Project.

Keywords: persistence, fstream, ofstream, ifstream, ios::app, struct, volatile memory, non-volatile disk, CSV

Introducción

Muchos programadores principiantes confunden la memoria RAM volátil, que se borra al finalizar el programa, con el disco no volátil, constituido por archivos de texto

persistentes. La Actividad 7 exige implementar ambas: la partida reside primero en memoria RAM como estructura y luego se escribe en disco para sobrevivir al cierre del programa.

Los objetivos son modelar con estructura, escribir con flujo de salida en modo adjunto sin sobrescribir y leer con flujo de entrada. El juego es el vehículo; la persistencia es el aprendizaje.

Revisión de la Literatura

La memoria RAM volátil es aquella cuyas variables desaparecen al terminar el proceso. La persistencia en disco es la escritura en almacenamiento secundario para permanencia. La biblioteca <*fstream*> gestiona flujos de archivo; el flujo de salida escribe del programa al archivo y el flujo de entrada lee del archivo al programa. La bandera *ios::app* indica adjuntar al final sin truncar, mientras que *cin.ignore()* limpia el salto de línea antes de *getline* (Stroustrup, 2013; cppreference.com, 2024).

Flujo	Dirección	Modo	Efecto sin modo
ofstream	RAM → Disco	ios::app	Sin app trunca y borra historial
ifstream	Disco → RAM	ios::in (predeterminado)	Lee secuencial con getline(',')

Nota. Flujos fstream y modos de apertura.

Método

La estructura */piedrapapelotijera* contiene *.gitattributes* para normalización de finales de línea, *src/main.cpp*, *src/historial_partidas.txt* que se genera en el primer guardado, *docs/CHANGELOG.md* y manuales. El programa principal presenta menú con ciclo *do/while* para opciones 1 a 3, inicializa la semilla aleatoria con *srand(time(0))* para la inteligencia artificial y define la estructura *Partida* con cinco campos.

La función *jugarPartida* solicita apodo con *getline* previo *cin.ignore()*, evalúa las reglas y llama a *guardarEnDisco*. Esta última abre *ofstream("src/historial_partidas.txt",*

ios::app), escribe en formato CSV y cierra. La función *leerHistorialDisco* abre *ifstream* y lee con *getline* delimitado por comas en ciclo. La compilación se ejecutó con *g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe*. Las pruebas consistieron en jugar una partida, cerrar la consola, reabrir y seleccionar la opción 2 para verificar la no volatilidad.

Resultados

Tras tres partidas, el archivo *historial_partidas.txt* contuvo tres líneas en formato CSV. Cerrar y reabrir el ejecutable no eliminó el archivo; la opción 2 listó las tres partidas con total de tres, demostrando persistencia. Sin *ios::app*, cada guardado truncaría el archivo y solo permanecería la última partida.

Figura 1. Menú y partida jugada (capturas/jugada_consola.png). Figura 2. Archivo en disco abierto (capturas/archivo_disco.png).

Discusión

La persistencia explica por qué el historial sobrevive al cierre: reside en disco, no en memoria RAM. La memoria RAM es volátil por diseño, rápida pero sin retención. Abrir sin *ios::app* emplea el modo truncado por defecto y borra el contenido previo; con modo adjunto, el puntero de escritura se posiciona al final. Sin *cin.ignore()*, el salto de línea remanente de *cin >> opcion* es consumido por *getline* como línea vacía, omitiendo la pregunta de apodo, defecto idéntico al de la Actividad 5.

Conclusión

El juego quedó funcional con persistencia verificada. El patrón estructura y flujo de archivo es directamente aplicable al Proyecto Integrador Anual: en lugar de jugadas, se almacenarían lecturas de sensores, registros de usuarios o stock en formato CSV. Como trabajo futuro se propone migrar de CSV a JSON o SQLite y agregar validación ante archivo corrupto.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

cppreference.com. (2024). *ios::app and file open modes*. https://en.cppreference.com/w/cpp/io/ios_base/openmode

cplusplus.com. (2024). *fstream — File streams*. https://cplusplus.com/reference/fstream/

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.